

第 18 章

用于大型程序的工具

导读

本章介绍了一些用于大型程序开发的语言特性，包括：

- 异常处理。
- 命名空间。
- 多重继承与虚继承。

本章的练习着重帮助读者理解异常的基本概念，练习异常捕获和异常处理的设计；理解命名空间和名字解析；理解多重继承和虚继承的概念，并进行相应的设计练习。

练习 18.1：在下列 throw 语句中异常对象的类型是什么？

(a) `range_error r("error");` (b) `exception *p = &r;`
 `throw r;` `throw *p;`

如果将(b)中的 throw 语句写成了 `throw p` 将发生什么情况？

【出题思路】

理解异常对象的类型的匹配方法和规则。

【解答】

(a) 异常对象 `r` 的类型是 `range_error`。

(b) 被抛出的异常对象是对指针 `p` 解引用的结果，其类型与 `p` 的静态类型相匹配，为 `exception`。若写成 `throw p`，则抛出的异常对象是 `exception*` 类型。

读者可尝试编译、运行这几条语句，观察系统的提示。

练习 18.2: 当在指定的位置发生了异常时将出现什么情况?

```
void exercise(int *b, int *e)
{
    vector<int> v(b, e);
    int *p = new int[v.size()];
    ifstream in("ints");
    // 此处发生异常
}
```

【出题思路】

深入理解异常发生时可能对程序已运行部分产生的影响。

【解答】

在 new 操作后发生的异常使得动态分配的数组没有被撤销, 从而造成内存泄漏。

练习 18.3: 要想让上面的代码在发生异常时能正常工作, 有两种解决方案。请描述这两种方法并实现它们。

【出题思路】

在理解异常发生可能造成的后果之后, 尝试使用 try-catch 方法和封装策略正确处理异常。

【解答】

方法一: 将有可能发生异常的代码放在 try 块中, 以便在异常发生时捕获异常。

```
void exercise(int *b, int *e)
{
    vector<int> v(b, e);
    int *p = new int[v.size()];
    try {
        ifstream in("ints");
        // 此处发生异常
    }
    catch {
        delete p;           // 释放数组
        // 进行其他处理
    }
    // ...
}
```

方法二: 定义一个类来封装数组的分配和释放, 以保证正确释放资源:

```
class Resource {
public:
    Resource(size_t sz) : r(new int[sz]) {}
    ~Resource() { if(r) delete r;}
private:
    int *r;
};
```

函数 exercise 相应修改为:

```
void exercise(int *b, int *e)
{
    vector<int> v(b, e);
```

```

    Resource res(v.size());
    ifstream in("ints");
    // exception occurs here
    // ...
}

```

注意，此处给出的 `Resource` 类非常简略，实际应用时，还需定义其他操作，包括复制构造函数、复制操作、解引用操作、箭头操作、下标操作等，以支持内置指针及数组的使用方式并保证自动删除 `Resource` 对象所引用的数组。另外，可将该 `Resource` 类定义为类模板，以支持多种数组元素类型。

练习 18.4：查看图 18.1（第 693 页）所示的继承体系，说明下面的 `try` 块有何错误并修改它。

```

try {
    // 使用 C++ 标准库
} catch(exception) {
    // ...
} catch(const runtime_error &re) {
    // ...
} catch(overflow_error eobj) { /* ... */ }

```

【出题思路】

理解异常处理中 `catch` 语句的匹配顺序。

【解答】

该 `try` 块中使用的 `exception`、`runtime_error` 及 `overflow_error` 是标准库中定义的异常类。它们是因继承而相关的：`runtime_error` 类继承 `exception` 类，`overflow_error` 类继承 `runtime_error` 类。在使用来自继承层次的异常时，`catch` 子句应该从最低派生类型到最高派生类型排序，以便派生类型的处理代码出现在其基类类型的 `catch` 之前，所以上述块中，`catch` 子句的顺序错误。

可修改为：

```

try {
    // 使用 C++ 标准库
} catch(overflow_error eobj) {
    // ...
} catch(const runtime_error &re) {
    // ...
} catch(exception) { /* ... */ }

```

练习 18.5：修改下面的 `main` 函数，使其能捕获图 18.1（第 693 页）所示的任何异常类型：

```

int main() {
    // 使用 C++ 标准库
}

```

处理代码应该首先打印异常相关的错误信息，然后调用 `abort`（定义在 `cstdlib`

头文件中) 终止 main 函数。

【出题思路】

本题练习捕获并打印所有类型的异常。

【解答】

解答:

```
int main()
{
    try {
        // 使用 C++ 标准库
    }
    catch(const exception &e) {
        cerr << e.what() << endl;
        abort();
    }
    return 0;
}
```

练习 18.6: 已知下面的异常类型和 catch 语句, 书写一个 throw 表达式使其创建的异常对象能被这些 catch 语句捕获:

- (a) class exceptionType { };
- catch(exceptionType *pet) { }
- (b) catch(...) { }
- (c) typedef int EXCPTYPE;
- catch(EXCPTYPE) { }

【出题思路】

理解使用 throw 时异常对象的类型一般需要与 catch 语句捕获的对象类型对应。

【解答】

- (a) throw new exceptionType(); // 动态创建一个异常对象并抛出其指针
- (b) throw 8; // 被抛出的表达式为任意类型
- (c) throw 11; // 被抛出的表达式为 int 类型即可

练习 18.7: 根据第 16 章的介绍定义你自己的 Blob 和 BlobPtr, 注意将构造函数写成函数 try 语句块。

【出题思路】

本题练习在实际应用中处理构造函数异常的常用方法: 使用 try 语句块。

【解答】

```
template <typename T> class Blob {
public:
    typedef T value_type;
    typedef typename std::vector<T>::size_type size_type;
    // 构造函数
    Blob();
    Blob(std::initializer_list<T> il) try:
        data{std::make_shared<std::vector<T>>(il)} {
```

```

/* 空函数体*/
    } catch(const std::bad_alloc &e) { handle_out_of_memory(e); }
    // Blob 中的元素数目
    size_type size() const { return data->size(); }
    bool empty() const { return data->empty(); }
    // 添加和删除元素
    void push_back(const T &t) { data->push_back(t); }
    // 移动版本, 参见 13.6.3 节 (第 548 页)
    void push_back(T &&t) { data->push_back(std::move(t)); }
    void pop_back();
    // 元素访问
    T& back();
    T& operator[](size_type i);           // 在 14.5 节中定义
private:
    std::shared_ptr<std::vector<T>> data;
    // 若 data[i] 无效, 则抛出 msg
    void check(size_type i, const std::string &msg) const;
};

template <typename T> class BlobPtr {
public:
    BlobPtr(): curr(0) { }
    BlobPtr(Blob<T> &a, size_t sz = 0) try:
        wptr(a.data), curr(sz) { }
    } catch(const std::bad_alloc &e) { handle_out_of_memory(e); }
    T& operator*() const
    { auto p = check(curr, "dereference past end");
      return (*p)[curr];           // (*p) 为本对象指向的 vector
    }
    // 递增和递减
    BlobPtr& operator++();           // 前置运算符
    BlobPtr& operator--();
private:
    // 若检查成功, check 返回一个指向 vector 的 shared_ptr
    std::shared_ptr<std::vector<T>>
        check(std::size_t, const std::string&) const;
    // 保存一个 weak_ptr, 表示底层 vector 可能被销毁
    std::weak_ptr<std::vector<T>> wptr;
    std::size_t curr;               // 数组中的当前位置
};

```

练习 18.8: 回顾你之前编写的各个类, 为它们的构造函数和析构函数添加正确的异常说明。如果你认为某个析构函数可能抛出异常, 尝试修改代码使得该析构函数不会抛出异常。

【出题思路】

练习 18.7 中已经说明了构造函数的异常处理方法, 练习 18.8 中着重理解析构函数的异常说明。

【解答】

```

class MyTest_Base
{
public:
    virtual ~MyTest_Base ()
    {
        cout << "开始准备销毁一个 MyTest_Base 类型的对象"<< endl;
        // 把异常完全封装在析构函数内部
        try
        {
            // 注意：在析构函数中抛出了异常
            throw std::exception("在析构函数中故意抛出一个异常，测试！");
        }
        catch(...) {}
    }

    void Func() throw()
    {
        throw std::exception("故意抛出一个异常，测试！");
    }

    void Other() {}
} // 重点是把可能发生异常的部分用 try 封装起来

```

练习 18.9：定义本节描述的书店程序异常类，然后为 `Sales_data` 类重新编写一个复合赋值运算符并令其抛出一个异常。

【出题思路】

本题练习书店应用程序中复合赋值运算符的编写和异常判断。

【解答】

```

Sales_data&
Sales_data::operator==(const Sales_data& rhs)
{
    if (isbn() != rhs.isbn())
        throw isbn_mismatch("wrong isbn", isbn(), rhs.isbn());
    units_sold += rhs.units_sold;
    revenue += rhs.revenue;
    return *this;
}

```

练习 18.10：编写程序令其对两个 ISBN 编号不相同的对象执行 `Sales_data` 的加法运算。为该程序编写两个不同的版本：一个处理异常、另一个不处理异常。观察并比较这两个程序的行为，用心体会当出现了一个未被捕获的异常时程序会发生什么情况。

【出题思路】

体会处理或不处理异常对程序的影响。下面是处理异常和不处理异常的主要程序片段。

【解答】

```

// 处理异常
Sales_data item1, item2, sum;
while (cin >> item1 >> item2) {           // 读取两条交易信息
    try {
        sum = item1 + item2;               // 计算它们的和
        return sum;
    } catch (const isbn_mismatch &e) {
        cerr << e.what() << ": left isbn(" << e.left
            << ") right isbn(" << e.right << ")" << endl;
    }
}

// 不处理异常
Sales_data item1, item2, sum;
while (cin >> item1 >> item2) {           // 读取两条交易信息
    sum = item1 + item2;                   // 计算它们的和
    return sum;
}

```

练习 18.11: 为什么 what 函数不应该抛出异常?

【出题思路】

深入理解 what 函数的作用和在异常处理中所处的重要位置。

【解答】

what 函数是在 catch 异常后用于提取异常基本信息的虚函数, what 函数是确保不会抛出任何异常的。如果 what 函数抛出了异常, 则会在新产生的异常中由于 what 函数继续产生异常, 将会产生抛出异常的死循环。所以 what 函数必须确保不抛出异常。

练习 18.12: 将你为之前各章练习编写的程序放置在各自的命名空间中。也就是说, 命名空间 chapter15 包含 Query 程序的代码, 命名空间 chapter10 包含 TextQuery 的代码; 使用这种结构重新编译 Query 代码示例。

【出题思路】

本题练习将不同程序放置在不同的命名空间中。

【解答】

将 Query 类以及 Query_base 类层次定义为命名空间 chapter15 的成员, 并相应修改主函数中的代码 (使用限定名引用这些类, 或者使用相关的 using 声明)。

练习 18.13: 什么时候应该使用未命名的命名空间?

【出题思路】

理解未命名的命名空间的作用和意义。

【解答】

通常，当需要声明局部于文件的实体时，可以使用未命名的命名空间，即在文件的最外层作用域中定义未命名的命名空间。

练习 18.14：假设下面的 `operator*` 声明是嵌套的命名空间 `mathLib::MatrixLib` 的一个成员：

```
namespace mathLib {
    namespace MatrixLib {
        class matrix { /* ... */ };
        matrix operator*
            (const matrix &, const matrix &);
        // ...
    }
}
```

请问你应该如何在全局作用域中声明该运算符？

【出题思路】

了解在嵌套的命名空间中如何声明全局的变量。

【解答】

将函数返回类型及函数名加上命名空间名字限定即可：

```
mathLib::MatrixLib::matrix mathLib::MatrixLib::operator* (const
matrix &, const matrix &)
{ /*...*/ }
```

练习 18.15：说明 `using` 指示与 `using` 声明的区别。

【出题思路】

深入了解 `using` 指示和 `using` 声明的特点和用处。

【解答】

一个 `using` 指示使得特定命名空间中的所有名字都成为可见的；而一个 `using` 声明只能引入特定命名空间中的一个成员。

练习 18.16：假定在下面的代码中标记为“位置 1”的地方是对于命名空间 `Exercise` 中所有成员的 `using` 声明，请解释代码的含义。如果这些 `using` 声明出现在“位置 2”又会怎样呢？将 `using` 声明变为 `using` 指示，重新回答之前的问题。

```
namespace Exercise {
    int ivar = 0;
    double dvar = 0;
    const int limit = 1000;
}
int ivar = 0;
// 位置 1
```



```

void manip() {
    // 位置 2
    double dvar = 3.1416;
    int iobj = limit + 1;
    ++ivar;
    ++::ivar;
}

```

【出题思路】

理解命名空间的声明和指示在主函数内和主函数外的差别。

【解答】

如果命名空间 Exercise 的所有成员的 using 声明放在标记为“位置 1”的地方，则 Exercise 中的成员在全局作用域中可见。using Exercise::ivar; 会导致 ivar 重复定义的编译错误，因为在全局作用域中也定义了一个同名变量（注意，由 using 声明引起的二义性错误在声明点检测）；而 manip 中的 double dvar = 3.1416; 声明了一个局部变量 dvar，在函数体作用域中它将屏蔽 Exercise::dvar; int iobj = limit + 1; 声明了一个局部变量 iobj，并用 Exercise::limit 加 1 的结果对其进行初始化。

如果命名空间 Exercise 的所有成员的 using 声明放在标记为“位置 2”的地方，则 manip 中的 double dvar = 3.1416; 属于对变量 dvar 的重复定义，会出现编译错误；int iobj = limit + 1; 声明了一个局部变量 iobj，并用 Exercise::limit 加 1 的结果对其进行初始化；++ivar; 访问到的是 Exercise::ivar，而 ++::ivar 访问的是全局变量 ivar。

如果命名空间 Exercise 的 using 指示放在标记为“位置 1”的地方，则 manip 中的 double dvar = 3.1416; 声明了一个局部变量 dvar，在函数体作用域中它将屏蔽 Exercise::dvar; int iobj = limit + 1; 声明了一个局部变量 iobj，并用 Exercise::limit 加 1 的结果对其进行初始化；++ivar; 访问到的是 Exercise::ivar，而 ++::ivar; 访问的是全局变量 ivar。

如果命名空间 Exercise 的 using 指示放在标记为“位置 2”的地方，则 Exercise 的成员看来好像是声明在全局作用域中的一样，manip 中的 double dvar = 3.1416; 声明了一个局部变量 dvar，在函数体作用域中它将屏蔽 Exercise::dvar;; int iobj = limit + 1; 声明了一个局部变量 iobj，并用 Exercise::limit 加 1 的结果对其进行初始化；++ivar; 出现二义性错误，因为编译器无法分辨是访问 Exercise::ivar，还是访问全局变量 ivar；而 ++::ivar 访问的是全局变量 ivar。

练习 18.17：实际编写代码检验你对上一题的回答是否正确。

【出题思路】

通过观察代码中相关变量的变化，深入了解命名空间的作用域。

【解答】

一共有4种情况，所以编程时需要编写4个对应的程序。

程序一：

```
namespace Exercise
{
    int ivar = 0;
    double dvar = 0;
    const int limit = 1000;
}

int ivar = 0;
// 位置1: 插入 using 声明
using Exercise::ivar;           // 编译错误: ivar 重复定义
using Exercise::dvar;
using Exercise::limit;

int main()
{
    // 位置2
    double dvar = 3.1416;       // 局部 dvar
    int iobj = limit + 1;       // Exercise::limit;
    ++ivar;                     // 二义性
    ++::ivar;                   // 二义性
    return 0;
}
```

程序二：

```
namespace Exercise
{
    int ivar = 0;
    double dvar = 0;
    const int limit = 1000;
}

int ivar = 0;
// 位置1

int main()
{
    // 位置2: 插入 using 声明
    using Exercise::ivar;
    using Exercise::dvar;
    using Exercise::limit;

    double dvar = 3.1416;       // 编译错误: 重复定义
    int iobj = limit + 1;       // Exercise::limit
    ++ivar;                     // Exercise::ivar
    ++::ivar;                   // 全局 ivar
    return 0;
}
```

程序三：

```
namespace Exercise
```

```

{
    int ivar = 0;
    double dvar = 0;
    const int limit = 1000;
}

int ivar = 0;
// 位置 1: 插入 using 指示
using namespace Exercise;

int main()
{
    // 位置 2
    double dvar = 3.1416;    // Exercise::dvar
    int iobj = limit + 1;    // Exercise::limit
    ++ivar;                  // 二义性
    ++::ivar;                // 二义性
    return 0;
}

```

程序四:

```

namespace Exercise
{
    int ivar = 0;
    double dvar = 0;
    const int limit = 1000;
}

int ivar = 0;
// 位置 1

int main()
{
    // 位置 2: 插入 using 指示
    using namespace Exercise;

    double dvar = 3.1416;    // Exercise::dvar
    int iobj = limit + 1;    // Exercise::limit
    ++ivar;                  // 二义性
    ++::ivar;                // 二义性
    return 0;
}

```

练习 18.18: 已知有下面的 `swap` 的典型定义(参与 13.3 节, 第 457 页), 当 `mem1` 是一个 `string` 时程序使用 `swap` 的哪个版本? 如果 `mem1` 是 `int` 呢? 说明在这两种情况下名字查找的过程。

```

void swap(T v1, T v2)
{
    using std::swap;
    swap(v1.mem1, v2.mem1);
}

```

```

        // 交换类型 T 的其他成员
    }

```

【出题思路】

深入理解同一函数在拥有不同类型的参数时名字查找的过程。

【解答】

如果 `mem1` 是 `string` 类型, 编译器除了在常规作用域中查找匹配的 `swap` 外, 还会查找 `string` 所属的命名空间中是否有 `string` 类型特定版本的 `swap` 函数。但对 `string` 而言, 找到的就是 `std::swap`, 完成两个字符串内容的交换。

若 `mem1` 是 `int` 类型, 由于 `int` 是内置类型, 没有特定版本的 `swap`, 只会在常规作用域中查找。由于 `using` 声明的作用, 最终会调用 `std::swap`, 完成两个 `int` 的交换。

练习 18.19: 如果对 `swap` 的调用形如 `std::swap(v1.mem1, v2.mem1)` 将发生什么情况?

【出题思路】

理解强制调用特定版本的函数时需要如何操作。

【解答】

将直接使用标准库版本的 `swap`, 而不会查找特定版本的 `swap` 或常规作用域中的其他 `swap`。

练习 18.20: 在下面的代码中, 确定哪个函数与 `compute` 调用匹配。列出所有候选函数和可行函数, 对于每个可行函数的实参与形参的匹配过程来说, 发生了哪种类型转换?

```

namespace primerLib {
    void compute();
    void compute(const void *);
}
using primerLib::compute;
void compute(int);
void compute(double, double = 3.4);
void compute(char*, char* = 0);
void f()
{
    compute(0);
}

```

如果将 `using` 声明置于函数 `f` 中 `compute` 的调用点之前将发生什么情况? 重新回答之前的那些问题。

【出题思路】

理解主函数内外 `using` 声明和重载的特性。

【解答】

全局作用域中声明的函数 `void compute(int)` 与 `compute` 函数的调用匹配。

候选函数：命名空间 `primerLib` 中声明的两个 `compute` 函数（因 `using` 声明使得它们在全局作用域中可见），以及全局作用域中声明的三个 `compute` 函数。

可行函数：因函数调用中给出的实参 `0` 为 `int` 型，所以可行函数为以下 4 个函数：

```
void compute(int)
void compute(double, double = 3.4)
void compute(char*, char* = 0)
primerLib 中声明的 void compute(const void*)
```

其中，第一个为完全匹配，第二个需要将实参隐式转换为 `double` 类型，第三个需要将实参隐式转换为 `char*` 类型，第四个需要将实参隐式转换为 `void*` 类型方可匹配，所以第一个为最佳匹配。

如果将 `using` 声明置于函数 `f` 中 `compute` 的调用点之前，则 `primerLib` 中声明的 `void compute(const void*)` 与 `compute` 函数的调用匹配。

候选函数：命名空间 `primerLib` 中声明的两个 `compute` 函数（因 `using` 声明使得它们在函数 `f` 的函数体作用域中可见，并屏蔽了全局作用域中的三个 `compute` 函数）。

可行函数：因函数调用中给出的实参 `0` 为 `int` 型，所以可行函数为 `primerLib` 中声明的 `void compute(const void*)`。需要将实参隐式转换为 `void*` 类型方可匹配。

练习 18.21：解释下列声明的含义，在它们当中存在错误吗？如果有，请指出来并说明错误的原因。

- (a) `class CADVehicle : public CAD, Vehicle { ... };`
- (b) `class DbList: public List, public List { ... };`
- (c) `class iostream: public istream, public ostream { ... };`

【出题思路】

理解多重继承的特性和规则。

【解答】

(b) 错误，在一个派生列表中，同一基类只能出现一次，这里 `List` 出现了两次。

练习 18.22：已知存在如下所示的类的继承体系，其中每个类都定义了一个默认构造函数：

```
class A { ... };
class B : public A { ... };
class C : public B { ... };
class X { ... };
class Y { ... };
class Z : public X, public Y { ... };
class MI : public C, public Z { ... };
```

对于下面的定义来说，构造函数的执行顺序是怎样的？

```
MI mi;
```

【出题思路】

了解多重继承中构造函数的构造顺序。

【解答】

构造函数的执行次序如下：

- (1) A 的构造函数
- (2) B 的构造函数
- (3) C 的构造函数
- (4) X 的构造函数
- (5) Y 的构造函数
- (6) Z 的构造函数
- (7) MI 的构造函数

练习 18.23：使用练习 18.22 的继承体系以及下面定义的类 D，同时假定每个类都定义了默认构造函数，请问下面的哪些类型转换是不被允许的？

```
class D : public X, public C { ... };
D *pd = new D;
(a) X *px = pd;      (b) A *pa = pd;
(c) B *pb = pd;      (d) C *pc = pd;
```

【出题思路】

了解多个基类情况下类型转换的特点。

【解答】(b)和(c)是不允许的。

因为 C 对 B 的继承是私有继承，使得在 D 中 B 的默认构造函数成为不可访问的，所以尽管存在从“D*”到“B*”以及从“D*”到“A*”的转换，但这些转换不可访问。

练习 18.24：在第 714 页，我们使用一个指向 Panda 对象的 Bear 指针进行了一系列调用，假设我们使用的是一个指向 Panda 对象的 ZooAnimal 指针将发生什么情况，请对这些调用语句逐一进行说明。

【出题思路】

深入理解基于指针类型的查找和调用顺序。

【解答】

如果使用 ZooAnimal 指针，则只能使用 ZooAnimal 类中定义的操作。

`pb->print(count);`，通过基类指针调用虚函数，使用动态绑定，pb 目前指向 Panda 对象，随意调用 `Panda::print(ostream&)`。

`pb->cuddle();`，因为 ZooAnimal 类中没有定义 `cuddle` 操作，所以该调用出错。

`pb->highlight();`，因为 ZooAnimal 类中没有定义 `highlight` 操作，所

以该调用出错。

`delete pb;`，因为 `ZooAnimal` 类中定义了虚析构函数，所以 `Panda` 类中的析构函数也是虚函数，因此 `delete pb;` 通过虚机制调用 `Panda` 析构函数。随着 `Panda` 析构函数的执行，依次调用 `Endangered`、`Bear` 和 `ZooAnimal` 的析构函数。

所以，通过指向 `Panda` 对象的 `Bear` 指针或 `ZooAnimal` 指针进行上述调用，将以同样方式确定函数调用。

练习 18.25：假设我们有两个基类 `Base1` 和 `Base2`，它们各自定义了一个名为 `print` 的虚成员和一个虚析构函数。从这两个基类中我们派生出下面的类，它们都重新定义了 `print` 函数：

```
class D1 : public Base1 { /* ... */ };
class D2 : public Base2 { /* ... */ };
class MI : public D1, public D2 { /* ... */ };
```

通过下面的指针，指出在每个调用中分别使用了哪个函数：

```
Base1 *pb1 = new MI;
Base2 *pb2 = new MI;
D1 *pd1 = new MI;
D2 *pd2 = new MI;
(a) pb1->print();      (b) pd1->print();      (c) pd2->print();
(d) delete pb2;        (e) delete pd1;      (f) delete pd2;
```

【出题思路】

深入理解虚机制和指针类型调用的过程。

【解答】

(a)、(b)和(c)均通过基类指针调用虚函数 `print`，这些基类指针当前都指向 `MI` 类对象，所以均调用 `MI::print()`；(d)、(e)和(f)均通过基类指针删除对象，这些基类指针当前都指向 `MI` 类对象，所以均通过虚机制调用 `MI` 析构函数，随着 `MI` 析构函数的执行，依次调用 `D2`、`Base2`、`D1` 和 `Base1` 的析构函数。

练习 18.26：已知如上所示的继承体系，下面对 `print` 的调用为什么是错误的？适当修改 `MI`，令其对 `print` 的调用可以编译通过并正确执行。

```
MI mi;
mi.print(42);
```

【出题思路】

了解通过类对象调用函数时需要注意的问题，并给出解决方案。

【解答】

因为 `mi.print(42);` 通过 `MI` 类对象调用 `print` 函数，编译器通过了名字查找，确定调用的是 `MI` 类中定义的 `print` 函数，但是 `MI` 类中定义的 `print` 函数需要 `std::vector<double>` 类型的参数，所以该调用是错误的。

改正：将 MI 中 print 的声明改为 `void print(int);`，该 print 调用即可正确编译和执行。

练习 18.27：已知如上所示的继承体系，同时假定为 MI 添加了一个名为 `foo` 的函数：

```
int ival;
double dval;
void MI::foo(double cval)
{
    int dval;
    // 练习中的问题发生在此处
}
```

- (a) 列出在 `MI::foo` 中可见的所有名字。
- (b) 是否存在某个可见的名字是继承自多个基类的？
- (c) 将 `Base1` 的 `dval` 成员与 `Derived` 的 `dval` 成员求和后赋给 `dval` 的局部实例。
- (d) 将 `MI::dvec` 的最后一个元素的值赋给 `Base2::fval`。
- (e) 将从 `Base1` 继承的 `cval` 赋给从 `Derived` 继承的 `sval` 的第一个字符。

【出题思路】

深入理解多重继承体系下类的作用域和可见域。

【解答】

- (a) `MI::foo` 中可见的名字有：`ival`、`dval`、`cval`、`sval`、`fval`、`dvec` 和 `print`
- (b) `Dval` 和 `print`
- (c) `dval = Base1::dval + Derived::dval;`
- (d) `fval = dvec.back();`
- (e) `sval[0] = cval;`

练习 18.28：已知存在如下的继承体系，在 `VMI` 类的内部哪些继承而来的成员无须前缀限定符就能直接访问？哪些必须有限定符才能访问？说明你的原因。

```
struct Base {
    void bar(int);           // 默认情况下是公有的
protected:
    int ival;
};
struct Derived1 : virtual public Base {
    void bar(char);          // 默认情况下是公有的
    void foo(char);
protected:
    char cval;
};
struct Derived2 : virtual public Base {
    void foo(int);           // 默认情况下是公有的
protected:
    int ival;
    char cval;
```



```
};
class VMI : public Derived1, public Derived2 { };
```

【出题思路】

了解在类继承中不同派生类的优先级顺序。

【解答】

从 VMI 类内部可以不加限定地访问继承成员 bar 和 ival。bar 在共享基类 Base 和派生类 Derived1 中都存在，但这只是在一条派生路径上，特定派生类实例的优先级高于共享基类实例，所以在 VMI 类内部不加限定地访问 bar，访问到的是 Derived1 中的 bar 实例。ival 在共享基类 Base 和派生类 Derived2 中都存在；同理，在 VMI 类内部不加限定地访问 ival，访问到的是 Derived2 中的 ival 实例。

继承成员 foo 和 cval 需要限定：二者在 Derived1 和 Derived2 中都存在，Derived1 和 Derived2 均为 Base 的派生类，访问优先级相同，所以，在 VMI 类内不加限定地访问 foo 和 cval，会出现二义性。

练习 18.29：已知有如下所示的类继承关系：

```
class Class { ... };
class Base : public Class { ... };
class D1 : virtual public Base { ... };
class D2 : virtual public Base { ... };
class MI : public D1, public D2 { ... };
class Final : public MI, public Class { ... };
```

- (a) 当作用于一个 Final 对象时，构造函数和析构函数的执行次序分别是什么？
- (b) 在一个 Final 对象中有几个 Base 部分？几个 Class 部分？
- (c) 下面的哪些赋值运算将造成编译错误？

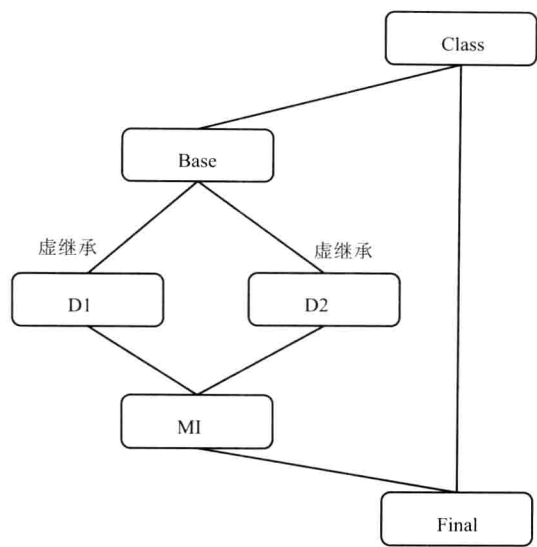
```
Base *pb;          Class *pc;          MI *pmi;          D2 *pd2;
(a) pb = new Class;      (b) pc = new Final;
(c) pmi = pb;            (d) pd2 = pmi;
```

【出题思路】

深入了解虚继承机制，析构函数和构造函数的执行顺序。

【解答】

- (a) 该类继承关系可表示如下：



如果定义 Final 对象，则创建该对象时按如下顺序调用构造函数：

```
Class();
Base();
D1();
D2();
MI();
Class();
Final();
```

注意，首先调用虚基类 Base 的构造函数（导致调用 Class 构造函数），然后按声明次序调用非虚基类的构造函数：首先是 MI（），它导致调用 D1（）和 D2（），然后是 Class（），最后调用 Final 类的构造函数。

撤销 Final 对象时调用析构函数的次序与调用构造函数的次序相反，如下所示：

```
~Final();
~Class();
~MI();
~D2();
~D1();
~Base();
~Class();
```

(b) 一个 Final 对象只有一个 Base 子对象，有两个 Class 子对象。

(c) 错误有

```
bp = new Class;    // 不能用派生类指针指向基类对象
pc = new Final;    // Class 是 Final 的一个二义基类
pmi = pb;          // 不能用指向基类的指针对指向派生类的指针进行赋值
```

练习 18.30：在 Base 中定义一个默认构造函数、一个拷贝构造函数和一个接受 int 形参的构造函数。在每个派生类中分别定义这三种构造函数，每个构造函数应该使用它的实参初始化其 Base 部分。

【出题思路】

本题练习派生类中不同类型构造函数的初始化方法。

【解答】

```
class Class {
    // ...
};

class Base : public Class{
    // ...
public:
    Base() : ival(0) {}
    Base(int i) : ival(i) {}
    Base(const Base& b) : ival(b.ival) {}
protected:
    int ival;
};

class D1: virtual public Base {
    // ...
public:
    D1() : Base(0) {};
    D1(int i) : Base(i) {};
    D1(const D1& d) : Base(d) {}
};

class D2:virtual public Base {
    // ...
public:
    D2() : Base(0) {}
    D2(int i) : Base(i) {}
    D2(const D2& d) : Base(d) {}
};

class M1 : public D1, public D2 {
    // ...
public:
    M1() : Base(0) {}
    M1(int i) : Base(i), D1(i), D2(i) {}
    M1(const M1& m) : Base(m), D1(m), D2(m) {}
};

class Final : public M1, public Class {
    // ...
public:
    Final() : Base(0) {}
    Final(int i) : Base(i), M1(i) {}
    Final(const Final& f) : Base(f), M1(f) {}
};
```